In [1]:

```
# === CELL A ===
# === CONFIG, DATA, RF (Institutional Setup) ===
import warnings; warnings.filterwarnings("ignore")
!pip install yfinance --upgrade
!pip install --upgrade pandas statsmodels scikit-learn matplotlib

import numpy as np, pandas as pd
import yfinance as yf
from pandas.tseries.offsets import BDay

# ---- User knobs (hedge fund perspective: single source of truth)
SYMS       = ["NVDA","AMZN","GOOGL","LLY","V","FCX","XOM","CEG","COST","LMT"

BENCHES    = {
    "SPY": "S&P 500",
    "DIA": "Dow Jones",
    "QQQ": "Nasdaq-100",
    "ACWI": "MSCI ACWI"
}

START      = "2010-01-01"   # longer history for multi-regime robustness
TRAIN_FRAC = 0.9            # use all data except the fixed OOS test window
TEST_DAYS  = None            # fixed 1-year out-of-sample window, or set to
THRESH_PCT = 0.2            # more realistic, less restrictive signal thresh
TX_COST    = 0.0002         # 0.02% per unit turnover (institutional executi
MOM_LKBK   = None            # classic 12-month momentum screen; set None to

ALL_TICKERS = list(dict.fromkeys(SYMS + list(BENCHES.keys()) + ["^IRX"]))  #

# ---- Download (OHLCV) & regularize
ohlcv = yf.download(ALL_TICKERS, start=START, auto_adjust=True, progress=Fal
H = ohlcv["High"].asfreq("B").ffill()
L = ohlcv["Low"].asfreq("B").ffill()
C = ohlcv["Close"].asfreq("B").ffill()
V = ohlcv["Volume"].asfreq("B").ffill()

# Core daily price frames
px       = C[SYMS]
bench_px = C[list(BENCHES.keys())]  # SPY, DIA, QQQ, ACWI
irx      = C["^IRX"]                # ^IRX is 13-week T-Bill yield in percen

# Convenience returns
px_ret      = px.pct_change().fillna(0.0)
bench_ret_m = bench_px.pct_change().fillna(0.0)
spy_ret     = bench_ret_m["SPY"]     # keep for CAPM bits downstream

# Typical price for VWAP (Shannon-style anchored VWAP on daily bars)
TP  = ((H + L + C) / 3.0)[SYMS]
VOL = V[SYMS]

# ---- Daily risk-free from ^IRX (13w T-Bill): % → decimal → compound to dai
rf_annual = (irx / 100.0).dropna()                  # annualized in decimal
rf_daily  = (1.0 + rf_annual)**(1/252.0) - 1.0      # daily compounding
rf_daily  = rf_daily.reindex(px.index).ffill().fillna(0.0)
```

```
# ---- Train/Test indices
if TEST_DAYS is not None:
    test_idx  = px.index[-TEST_DAYS:]
    train_idx = px.index.difference(test_idx)
else:
    cut       = int(len(px) * TRAIN_FRAC)
    train_idx = px.index[:cut]
    test_idx  = px.index[cut:]

print(f"Universe={len(SYMS)}; Train={train_idx[0].date()}→{train_idx[-1].dat
      f"Test={test_idx[0].date()}→{test_idx[-1].date()} | THRESH={THRESH_PCT
```

```
Collecting yfinance
  Using cached yfinance-0.2.65-py2.py3-none-any.whl.metadata (5.8 kB)
Requirement already satisfied: pandas>=1.3.0 in /opt/conda/lib/python3.12/si
te-packages (from yfinance) (2.3.1)
Requirement already satisfied: numpy>=1.16.5 in /opt/conda/lib/python3.12/si
te-packages (from yfinance) (2.2.6)
Requirement already satisfied: requests>=2.31 in /opt/conda/lib/python3.12/s
ite-packages (from yfinance) (2.32.4)
Requirement already satisfied: multitasking>=0.0.7 in ./.local/lib/python3.1
2/site-packages (from yfinance) (0.0.12)
Requirement already satisfied: platformdirs>=2.0.0 in /opt/conda/lib/python
3.12/site-packages (from yfinance) (4.3.8)
Requirement already satisfied: pytz>=2022.5 in /opt/conda/lib/python3.12/sit
e-packages (from yfinance) (2025.2)
Requirement already satisfied: frozendict>=2.3.4 in /opt/conda/lib/python3.1
2/site-packages (from yfinance) (2.4.6)
Requirement already satisfied: peewee>=3.16.2 in ./.local/lib/python3.12/sit
e-packages (from yfinance) (3.18.2)
Requirement already satisfied: beautifulsoup4>=4.11.1 in /opt/conda/lib/pyth
on3.12/site-packages (from yfinance) (4.13.4)
Requirement already satisfied: curl_cffi>=0.7 in ./.local/lib/python3.12/sit
e-packages (from yfinance) (0.13.0)
Requirement already satisfied: protobuf>=3.19.0 in /opt/conda/lib/python3.1
2/site-packages (from yfinance) (5.28.3)
Requirement already satisfied: websockets>=13.0 in ./.local/lib/python3.12/s
ite-packages (from yfinance) (15.0.1)
Requirement already satisfied: soupsieve>1.2 in /opt/conda/lib/python3.12/si
te-packages (from beautifulsoup4>=4.11.1->yfinance) (2.7)
Requirement already satisfied: typing-extensions>=4.0.0 in /opt/conda/lib/py
thon3.12/site-packages (from beautifulsoup4>=4.11.1->yfinance) (4.14.1)
Requirement already satisfied: cffi>=1.12.0 in /opt/conda/lib/python3.12/sit
e-packages (from curl_cffi>=0.7->yfinance) (1.17.1)
Requirement already satisfied: certifi>=2024.2.2 in /opt/conda/lib/python3.1
2/site-packages (from curl_cffi>=0.7->yfinance) (2025.7.14)
Requirement already satisfied: pycparser in /opt/conda/lib/python3.12/site-p
ackages (from cffi>=1.12.0->curl_cffi>=0.7->yfinance) (2.22)
Requirement already satisfied: python-dateutil>=2.8.2 in /opt/conda/lib/pyth
on3.12/site-packages (from pandas>=1.3.0->yfinance) (2.9.0.post0)
Requirement already satisfied: tzdata>=2022.7 in /opt/conda/lib/python3.12/s
ite-packages (from pandas>=1.3.0->yfinance) (2025.2)
Requirement already satisfied: six>=1.5 in /opt/conda/lib/python3.12/site-pa
ckages (from python-dateutil>=2.8.2->pandas>=1.3.0->yfinance) (1.17.0)
Requirement already satisfied: charset_normalizer<4,>=2 in /opt/conda/lib/py
thon3.12/site-packages (from requests>=2.31->yfinance) (3.4.2)
Requirement already satisfied: idna<4,>=2.5 in /opt/conda/lib/python3.12/sit
e-packages (from requests>=2.31->yfinance) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in /opt/conda/lib/python3.
12/site-packages (from requests>=2.31->yfinance) (2.5.0)
Using cached yfinance-0.2.65-py2.py3-none-any.whl (119 kB)
Installing collected packages: yfinance
Successfully installed yfinance-0.2.65
Requirement already satisfied: pandas in /opt/conda/lib/python3.12/site-pack
ages (2.3.1)
Collecting pandas
  Using cached pandas-2.3.2-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_
x86_64.whl.metadata (91 kB)
```

```
Requirement already satisfied: statsmodels in /opt/conda/lib/python3.12/site
-packages (0.14.5)
Requirement already satisfied: scikit-learn in /opt/conda/lib/python3.12/sit
e-packages (1.7.1)
Requirement already satisfied: matplotlib in /opt/conda/lib/python3.12/site-
packages (3.10.3)
Collecting matplotlib
  Using cached matplotlib-3.10.6-cp312-cp312-manylinux2014_x86_64.manylinux_
2_17_x86_64.whl.metadata (11 kB)
Requirement already satisfied: numpy>=1.26.0 in /opt/conda/lib/python3.12/si
te-packages (from pandas) (2.2.6)
Requirement already satisfied: python-dateutil>=2.8.2 in /opt/conda/lib/pyth
on3.12/site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /opt/conda/lib/python3.12/sit
e-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /opt/conda/lib/python3.12/s
ite-packages (from pandas) (2025.2)
Requirement already satisfied: scipy!=1.9.2,>=1.8 in /opt/conda/lib/python3.
12/site-packages (from statsmodels) (1.16.0)
Requirement already satisfied: patsy>=0.5.6 in /opt/conda/lib/python3.12/sit
e-packages (from statsmodels) (1.0.1)
Requirement already satisfied: packaging>=21.3 in /opt/conda/lib/python3.12/
site-packages (from statsmodels) (25.0)
Requirement already satisfied: joblib>=1.2.0 in /opt/conda/lib/python3.12/si
te-packages (from scikit-learn) (1.5.1)
Requirement already satisfied: threadpoolctl>=3.1.0 in /opt/conda/lib/python
3.12/site-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: contourpy>=1.0.1 in /opt/conda/lib/python3.1
2/site-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /opt/conda/lib/python3.12/sit
e-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /opt/conda/lib/python3.1
2/site-packages (from matplotlib) (4.59.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /opt/conda/lib/python3.1
2/site-packages (from matplotlib) (1.4.8)
Requirement already satisfied: pillow>=8 in /opt/conda/lib/python3.12/site-p
ackages (from matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /opt/conda/lib/python3.1
2/site-packages (from matplotlib) (3.2.3)
Requirement already satisfied: six>=1.5 in /opt/conda/lib/python3.12/site-pa
ckages (from python-dateutil>=2.8.2->pandas) (1.17.0)
Using cached pandas-2.3.2-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x8
6_64.whl (12.0 MB)
Using cached matplotlib-3.10.6-cp312-cp312-manylinux2014_x86_64.manylinux_2_
17_x86_64.whl (8.7 MB)
Installing collected packages: pandas, matplotlib
  Attempting uninstall: pandas
    Found existing installation: pandas 2.3.1
    Uninstalling pandas-2.3.1:
      Successfully uninstalled pandas-2.3.1
  Attempting uninstall: matplotlib━━━━━━━━━━━━━━━━━━━━ 0/2 [pandas]
    Found existing installation: matplotlib 3.10.3[32m0/2 [pandas]
    Uninstalling matplotlib-3.10.3:━━━━━━━━━━━━━━━━━━━━ 0/2 [pandas]
      Successfully uninstalled matplotlib-3.10.3━━━━━━━━━━━━━━━━━━━━ 1/2 [matp
lotlib]
   ━━━━━━━━━━━━━━━━━━━
```

```
Successfully installed matplotlib-3.10.6 pandas-2.3.2
Universe=10; Train=2010-01-04→2024-02-12 | Test=2024-02-13→2025-09-08 | THRE
SH=0.2% | TX=0.02%
```

In [2]:

```
# === Cell B ===
# Robust ARIMA: pick order once, fast periodic refits, no invalid optimizers
from statsmodels.tsa.arima.model import ARIMA as SM_ARIMA
from statsmodels.stats.diagnostic import acorr_ljungbox, het_arch
from statsmodels.stats.stattools import jarque_bera
from IPython.display import display
import warnings
import numpy as np
import pandas as pd

# Silence only convergence warnings (keep real errors)
try:
    from statsmodels.base.model import ConvergenceWarning as SMConvergenceWa
    warnings.filterwarnings("ignore", category=SMConvergenceWarning)
except Exception:
    pass

# ---- Tunables (you can loosen/tighten these)
CANDIDATES            = [(1,1,0), (0,1,1), (1,1,1), (2,1,0)]  # compact, rob
TREND                 = "n"        # no trend for d=1 models
MIN_TRAIN_OBS         = 60
MAX_TRAIN_OBS_USED    = 1000       # cap training history to last N obs for
REFIT_EVERY           = 5         # refit every N test days (lighter -> much
MAXITER               = 120        # optimizer cap
ALPHA                 = 0.05

def _fit_arima_levels(y, order, trend=TREND):
    """Statsmodels ARIMA with valid API (no 'method' arg); use method_kwargs
    y_use = y.iloc[-MAX_TRAIN_OBS_USED:] if len(y) > MAX_TRAIN_OBS_USED else
    mod = SM_ARIMA(
        y_use, order=order, trend=trend,
        enforce_stationarity=False, enforce_invertibility=False
    )
    # Do NOT pass 'method' here; ARIMA validates estimator names strictly.
    res = mod.fit(method_kwargs={"maxiter": MAXITER})
    return res

def _pick_order_once(y_tr):
    """Choose order by AIC on the training window once per ticker."""
    y_use = y_tr.iloc[-MAX_TRAIN_OBS_USED:] if len(y_tr) > MAX_TRAIN_OBS_USE
    best, best_aic = None, np.inf
    for od in CANDIDATES:
        try:
            r = _fit_arima_levels(y_use, od, TREND)
            aic = getattr(r, "aic", np.inf)
            if np.isfinite(aic) and aic < best_aic:
                best, best_aic = (od, r), aic
        except Exception:
            continue
    # Fallback: if nothing worked, just return a default order and None resu
    # downstream refits will try again with the chosen order.
    return best if best is not None else ((1,1,0), None)
```

```
def _drift_fallback(hist_log, h):
    """
    Simple drift-on-log-returns fallback with 95% CI:
    mean ± 1.96*std of ΔlogP over the last 60 bars, integrated back to level
    """
    yd = hist_log.diff().dropna()
    if len(yd) == 0:
        last = float(np.exp(hist_log.iloc[-1]))
        return last, last, last
    mu  = yd.tail(60).mean()
    sd  = yd.tail(60).std(ddof=1) if len(yd.tail(60)) > 1 else yd.std(ddof=1
    mu_cum = mu * h
    sd_cum = (sd * np.sqrt(h)) if np.isfinite(sd) else 0.0
    base = float(hist_log.iloc[-1])
    fc = np.exp(base + mu_cum)
    lo = np.exp(base + mu_cum - 1.96*sd_cum)
    hi = np.exp(base + mu_cum + 1.96*sd_cum)
    return float(fc), float(lo), float(hi)

def fit_forecast_quant(price_df, train_idx, test_idx):
    fc_map, lo_map, hi_map, act_map = {}, {}, {}, {}
    metrics_rows, stats_rows, latest_rows = [], [], []

    for sym in price_df.columns:
        y_all = np.log(price_df[sym].replace(0, np.nan).dropna())
        y_tr  = y_all.reindex(train_idx).dropna()
        y_te  = y_all.reindex(test_idx).dropna()
        if len(y_tr) < MIN_TRAIN_OBS or len(y_te) == 0:
            print(f"[SKIP] {sym}: insufficient history."); continue

        # ---- Pick order once on the train slice
        (order, init_res) = _pick_order_once(y_tr)
        model_mode = "ARIMA"
        converged_flag = np.nan
        aic = bic = llf = np.nan

        # If we got an initial fit, log diagnostics; else we'll log after fi
        if init_res is not None:
            resid = init_res.resid.dropna()
            try: aic = init_res.aic; bic = init_res.bic; llf = init_res.llf
            except Exception: pass
            try: lb_p  = acorr_ljungbox(resid, lags=[10], return_df=True)["l
            except Exception: lb_p = np.nan
            try: jb_p = jarque_bera(resid)[1]
            except Exception: jb_p = np.nan
            try: arch_p = het_arch(resid, nlags=5)[1]
            except Exception: arch_p = np.nan
            try:
                npar  = int(len(init_res.params)) if hasattr(init_res, "para
                tvals = getattr(init_res, "tvalues", None)
                sig_k = int(np.sum(np.abs(tvals) > 1.96)) if tvals is not No
            except Exception:
                npar, sig_k = np.nan, np.nan
            try:
                converged_flag = bool(getattr(getattr(init_res, "mle_retvals
```

```
            except Exception:
                converged_flag = np.nan
        else:
            lb_p = jb_p = arch_p = npar = sig_k = np.nan

        stats_rows.append({
            "Ticker": sym, "Mode": model_mode, "BestOrder": order, "BestTrer
            "Train_N": len(y_tr), "AIC": aic, "BIC": bic, "LLF": llf,
            "LjungBox_p(10)": lb_p, "JarqueBera_p": jb_p, "ARCH_LM_p(5)": ar
            "Params": npar, "SigParams(|t|>1.96)": sig_k, "Converged": conve
        })

        # ---- Walk-forward OOS with periodic refits (fixed order)
        fc_list, lo_list, hi_list, act_list, dates = [], [], [], [], []
        last_refit_step = None
        refit_res = init_res

        for step, d in enumerate(test_idx, start=1):
            if d not in y_all.index: continue
            hist = y_all.loc[:d].iloc[:-1]
            if len(hist) < MIN_TRAIN_OBS: continue

            # schedule a refit
            if (last_refit_step is None) or ((step - last_refit_step) >= REF
                try:
                    refit_res = _fit_arima_levels(hist, order, TREND)
                    last_refit_step = step
                    h = 1
                except Exception:
                    refit_res = None  # force fallback below
                    last_refit_step = step
                    h = 1
            else:
                h = step - last_refit_step + 1

            # forecast h-steps ahead
            try:
                if refit_res is not None:
                    pred   = refit_res.get_forecast(steps=h)
                    mean_h = pred.predicted_mean.iloc[-1]
                    ci_h   = pred.conf_int(alpha=ALPHA).iloc[-1]
                    fc_val = float(np.exp(mean_h))
                    lo_val = float(np.exp(ci_h.iloc[0]))
                    hi_val = float(np.exp(ci_h.iloc[1]))
                else:
                    # fallback forecast from drift on log-returns
                    fc_val, lo_val, hi_val = _drift_fallback(hist, h)
                act_val = float(np.exp(y_all.loc[d]))
            except Exception:
                # total fallback if anything else goes wrong
                fc_val, lo_val, hi_val = _drift_fallback(hist, h)
                act_val = float(np.exp(y_all.loc[d]))

            fc_list.append(fc_val); lo_list.append(lo_val); hi_list.append(h
            act_list.append(act_val); dates.append(d)
```

```
        if not dates:
            print(f"[WARN] {sym}: produced no OOS forecasts."); continue

        idx = pd.DatetimeIndex(dates)
        fc_s  = pd.Series(fc_list, index=idx, name=sym)
        lo_s  = pd.Series(lo_list, index=idx, name=sym)
        hi_s  = pd.Series(hi_list, index=idx, name=sym)
        act_s = pd.Series(act_list, index=idx, name=sym)

        fc_map[sym], lo_map[sym], hi_map[sym], act_map[sym] = fc_s, lo_s, hi

        # OOS errors
        common = act_s.index.intersection(fc_s.index)
        err  = (act_s.loc[common] - fc_s.loc[common])
        mae  = err.abs().mean()
        rmse = np.sqrt((err**2).mean())
        mape = (err.abs() / act_s.loc[common]).mean() * 100.0
        metrics_rows.append({"Ticker": sym, "MAE": mae, "RMSE": rmse, "MAPE_

        last_day = common.max()
        latest_rows.append({
            "Ticker": sym, "Date": last_day.date(),
            "Forecast": fc_s.loc[last_day], "Lo95": lo_s.loc[last_day],
            "Hi95": hi_s.loc[last_day], "Actual": act_s.loc[last_day],
            "Error_%": 100.0 * (fc_s.loc[last_day] - act_s.loc[last_day]) /
        })

    # ---- Pack outputs
    fc_df  = pd.DataFrame(fc_map).sort_index()
    lo_df  = pd.DataFrame(lo_map).sort_index()
    hi_df  = pd.DataFrame(hi_map).sort_index()
    act_df = pd.DataFrame(act_map).sort_index()

    metrics_df = pd.DataFrame(metrics_rows)
    metrics_df = metrics_df.set_index("Ticker").sort_values("MAPE_%") if not

    stats_df = pd.DataFrame(stats_rows)
    arima_stats_df = stats_df.set_index("Ticker").sort_values(["AIC","BIC"],
        columns=["Mode","BestOrder","BestTrend","AIC","BIC","LLF","LjungBox_
    )

    latest_fc_df = pd.DataFrame(latest_rows)
    latest_fc_df = latest_fc_df.set_index("Ticker").sort_index() if not late

    return fc_df, lo_df, hi_df, act_df, metrics_df, arima_stats_df, latest_f

# ---- Run it
fc_df, lo_df, hi_df, act_df, metrics_df, arima_stats_df, latest_fc_df = fit_
print("=== ARIMA Model Stats (training-fit) ==="); display(arima_stats_df)
print("=== OOS Forecast Error Metrics (test window) ==="); display(metrics_d
print("=== Latest Forecast vs Actual (test window) ==="); display(latest_fc_
```

```
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
```

```
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
```

```
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
```

```
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
```

```
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
```

```
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
=== ARIMA Model Stats (training-fit) ===
```

```
/opt/conda/lib/python3.12/site-packages/statsmodels/base/model.py:607: Conve
rgenceWarning: Maximum Likelihood optimization failed to converge. Check mle
_retvals
  warnings.warn("Maximum Likelihood optimization failed to "
```

Ticker	Mode	BestOrder	BestTrend	Train_N	AIC	BIC	LLF
LMT	ARIMA	(1, 1, 0)	n	3681	-5668.901071	-5659.089565	2836.450536
COST	ARIMA	(1, 1, 0)	n	3681	-5657.129351	-5647.317845	2830.564676
V	ARIMA	(1, 1, 0)	n	3681	-5519.304611	-5509.493105	2761.652306
LLY	ARIMA	(1, 1, 0)	n	3681	-5107.023074	-5097.211568	2555.511537
GOOGL	ARIMA	(1, 1, 0)	n	3681	-5038.724893	-5028.913387	2521.362447
XOM	ARIMA	(1, 1, 0)	n	3681	-4897.934462	-4888.122955	2450.967231
AMZN	ARIMA	(1, 1, 0)	n	3681	-4712.545500	-4702.733994	2358.272750
FCX	ARIMA	(1, 1, 0)	n	3681	-4185.412129	-4175.600623	2094.706065
NVDA	ARIMA	(1, 1, 0)	n	3681	-4083.382288	-4073.570782	2043.691144
CEG	ARIMA	(1, 1, 0)	n	539	-2515.348993	-2506.776996	1259.674496

```
=== OOS Forecast Error Metrics (test window) ===
```

Ticker	MAE	RMSE	MAPE_%
COST	14.357668	20.044904	1.586796
V	5.108102	7.143087	1.647516
LMT	8.738883	13.713443	1.848615
XOM	2.034731	2.672115	1.866695
AMZN	4.531853	6.137674	2.293928
GOOGL	4.016817	5.674838	2.330350
LLY	25.069559	34.571346	3.161773
FCX	1.339244	1.816822	3.181717
NVDA	4.963396	6.419283	4.126992
CEG	10.207457	14.324908	4.355818

```
=== Latest Forecast vs Actual (test window) ===
```

Ticker	Date	Forecast	Lo95	Hi95	Actual	Error_%
AMZN	2025-09-08	229.000000	207.370352	252.885716	235.839996	-2.900270
CEG	2025-09-08	307.980011	269.107444	352.467720	298.820007	3.065392
COST	2025-09-08	943.320007	882.667273	1008.140511	971.849976	-2.935635
FCX	2025-09-08	44.400002	39.459409	49.959191	46.660000	-4.843545
GOOGL	2025-09-08	212.910004	195.180369	232.250149	234.039993	-9.028367
LLY	2025-09-08	732.580017	668.416209	802.903152	738.640015	-0.820426
LMT	2025-09-08	452.329987	424.978288	481.442047	459.109985	-1.476770
NVDA	2025-09-08	174.179993	150.718407	201.293727	168.309998	3.487609
V	2025-09-08	351.779999	330.895214	373.982948	342.299988	2.769504
XOM	2025-09-08	114.290001	105.982832	123.248305	109.849998	4.041878

In [8]:

```
# === Cell C ===

# === VWAP UTILITIES (Anchored or Rolling) — FIXED ===

def anchored_or_rolling_vwap(tp_df, vol_df, *, anchor_index=None, lookback=2
    """
    Daily VWAP proxy using typical price (TP) & volume (VOL).
    - If lookback is None: ANCHORED VWAP from the first date in `anchor_inde
    - If lookback is int:  ROLLING VWAP over `lookback` business days.
    Returns a DataFrame aligned to tp_df/vol_df index & columns.
    """
    eps = 1e-12
    tp_df = tp_df.copy()
    vol_df = vol_df.copy()

    if lookback is not None:
        # Rolling (windowed) VWAP
        num = (tp_df * vol_df).rolling(lookback, min_periods=1).sum()
        den = vol_df.rolling(lookback, min_periods=1).sum().clip(lower=eps)
        return num / den

    # --- Anchored VWAP from anchor start ---
    if anchor_index is None or len(anchor_index) == 0:
        raise ValueError("Provide anchor_index (e.g., test_idx) when lookbac

    start_date = anchor_index[0]

    # Cumulative sums from the beginning…
    cum_num = (tp_df * vol_df).cumsum()
    cum_den = vol_df.cumsum().clip(lower=eps)

    # …subtract the cumulative base at the day before the anchor
```

```
    # If the anchor is the first row, base is zeros.
    try:
        # position of start_date in the index (may raise if not found)
        pos = tp_df.index.get_loc(start_date)
    except KeyError:
        # If start_date not in index, align to the next valid index point
        pos = tp_df.index.searchsorted(start_date)

    if pos > 0:
        base_num = cum_num.iloc[pos - 1]
        base_den = cum_den.iloc[pos - 1]
    else:
        # No prior day; use zeros
        base_num = cum_num.iloc[0] * 0.0
        base_den = cum_den.iloc[0] * 0.0

    adj_num = (cum_num - base_num)
    adj_den = (cum_den - base_den).clip(lower=eps)
    vwap = adj_num / adj_den

    # For dates before start_date, VWAP is NaN (not defined yet)
    vwap.loc[vwap.index < start_date] = np.nan
    return vwap

# --- Choose your VWAP style:
VWAP_LOOKBACK = None      # None → Anchored at test start; or set int (e.g.,
VWAP = anchored_or_rolling_vwap(TP, VOL, anchor_index=test_idx, lookback=VWA

# Keep a test-window slice handy
VWAP_TEST = VWAP.reindex(test_idx)
PX_TEST   = px.reindex(test_idx)
```

In [9]:

```
# === Cell D ===
# Scenarios & Benchmarks (semi-annual rebal, $100k) — now *charging TX_COST*
import matplotlib.pyplot as plt

INITIAL_CAPITAL  = 100_000.0
REBAL_MONTHS     = 6
TRADE_WEIGHT_BPS = 1.0
EPS              = 1e-12

def make_rebalance_dates(idx, months=6):
    dates = [idx[0]]
    last = idx[0]
    while True:
        m = (last.month - 1 + months) % 12 + 1
        y = last.year + (last.month - 1 + months) // 12
        day = min(last.day, 28)
        tentative = pd.Timestamp(year=y, month=m, day=day)
        pos = idx.searchsorted(tentative)
        if pos >= len(idx): break
        aligned = idx[pos]
        if aligned != dates[-1]: dates.append(aligned)
        last = aligned
    return pd.DatetimeIndex(dates)
```

```
REBAL_DATES = make_rebalance_dates(px.index, months=REBAL_MONTHS)

def fetch_shares_outstanding(symbols):
    so = {}
    for s in symbols:
        try:
            info = yf.Ticker(s).fast_info
            shares = getattr(info, "shares", None)
            if shares is None:
                shares = yf.Ticker(s).info.get("sharesOutstanding", None)
            if shares is not None and shares > 0:
                so[s] = float(shares)
        except Exception:
            pass
    return so

SHARES_OUT = fetch_shares_outstanding(SYMS)
shares_series_full = pd.Series(SHARES_OUT, index=SYMS, dtype=float)

def target_equal_weight(date, prices_row):
    n = prices_row.notna().sum()
    w = pd.Series(0.0, index=prices_row.index)
    if n > 0: w.loc[prices_row.notna()] = 1.0 / n
    return w

def make_target_cap_weight(prices_df, shares_series, mode="dynamic", anchor_
    shares_series = shares_series.reindex(prices_df.columns)
    if mode == "anchored":
        if anchor_date is None: anchor_date = REBAL_DATES[0]
        anchor_prices = prices_df.loc[anchor_date]
        caps = (anchor_prices * shares_series)
        weights = pd.Series(0.0, index=prices_df.columns)
        if caps.dropna().sum() > 0:
            weights.loc[caps.dropna().index] = caps.dropna() / caps.dropna()
        miss = shares_series.isna() & anchor_prices.notna()
        if miss.any():
            leftover = max(0.0, 1.0 - weights.sum()); k = int(miss.sum())
            if k > 0: weights.loc[miss] = leftover / k
        weights = weights.where(anchor_prices.notna(), 0.0).fillna(0.0)
        def _anch(date, prices_row): return weights.where(prices_row.notna()
        return _anch
    else:
        def _dyn(date, prices_row):
            caps = (prices_row * shares_series)
            weights = pd.Series(0.0, index=prices_row.index)
            if caps.dropna().sum() > 0:
                weights.loc[caps.dropna().index] = caps.dropna() / caps.drop
            miss = caps.isna() & prices_row.notna()
            if miss.any():
                leftover = max(0.0, 1.0 - weights.sum()); k = int(miss.sum()
                if k > 0: weights.loc[miss] = leftover / k
            return weights.where(prices_row.notna(), 0.0).fillna(0.0)
        return _dyn

CAPW_MODE = "anchored"
target_cap_weight = make_target_cap_weight(px, shares_series_full, mode=CAPW
```

```
def simulate_rebalanced_portfolio(prices, rebalance_dates, target_fn, initia
                                  trade_weight_bps=1.0, tx_cost=0.0, eps=1e-
    idx, rets = prices.index, prices.pct_change().fillna(0.0)
    pos_val  = pd.DataFrame(0.0, index=idx, columns=prices.columns)
    port_val = pd.Series(index=idx, dtype=float)
    weights  = pd.DataFrame(0.0, index=idx, columns=prices.columns)

    t0 = idx[0]
    w0 = target_fn(t0, prices.loc[t0]).fillna(0.0)
    port_val.iloc[0] = initial_capital
    pos_val.loc[t0]  = w0 * initial_capital
    weights.loc[t0]  = w0

    rebalance_log = []

    for i in range(1, len(idx)):
        d, prev = idx[i], idx[i-1]
        pos_val.loc[d] = pos_val.loc[prev] * (1.0 + rets.loc[d])
        pv = float(pos_val.loc[d].sum())
        port_val.iloc[i] = pv if pv > 0 else 0.0
        w_pre = (pos_val.loc[d] / max(pv, eps)).fillna(0.0)

        if d in rebalance_dates:
            w_tgt = target_fn(d, prices.loc[d]).fillna(0.0)
            dw = (w_tgt - w_pre)
            gross_turnover = float(dw.abs().sum())
            cost = tx_cost * gross_turnover * pv
            pv_after = max(pv - cost, 0.0)
            pos_val.loc[d] = w_tgt * pv_after
            weights.loc[d] = w_tgt
            thresh = trade_weight_bps / 10000.0
            trades_today = int((dw.abs() > max(thresh, eps)).sum())
            rebalance_log.append({"date": d, "trades": trades_today,
                                  "gross_turnover": gross_turnover, "cost":
        else:
            weights.loc[d] = w_pre

    total_trades = sum(x["trades"] for x in rebalance_log)
    rebalance_df = pd.DataFrame(rebalance_log).set_index("date") if rebalanc
    return port_val, weights, total_trades, rebalance_df

# Build scenarios (now pay TX_COST)
eq_val, eq_w, eq_trades, eq_reb = simulate_rebalanced_portfolio(px, REBAL_DA
                                                                INITIAL_CAP
cw_val, cw_w, cw_trades, cw_reb = simulate_rebalanced_portfolio(px, REBAL_DA
                                                                INITIAL_CAP

# Benchmarks to $100k
bench_curves = (1.0 + bench_ret_m).loc[eq_val.index].cumprod() * INITIAL_CAP
bench_curves.columns = [f"{k} ({BENCHES[k]})" for k in bench_curves.columns]
```

In [10]:

```
# === Cell E ===
# Fundamentals layer: static snapshot via yfinance 'info' (robust to missing
# Used as an extra gate for S1 and S2 (NOT for S3 base case).
```

```
USE_FUNDAMENTALS = True        # set False to disable fundamentals gate
MIN_FUND_SCORE   = 3           # fallback: hard cutoff if TOP_FUND_FRAC is N
TOP_FUND_FRAC    = 0.70        # keep top 70% by fundamentals score (set to

def fetch_fundamentals(symbols):
    rows = []
    for s in symbols:
        info = {}
        try:
            info = yf.Ticker(s).info or {}
        except Exception:
            info = {}
        rows.append({
            "Ticker": s,
            "profitMargins":    info.get("profitMargins"),
            "operatingMargins": info.get("operatingMargins"),
            "returnOnEquity":   info.get("returnOnEquity"),   # fraction
            "revenueGrowth":    info.get("revenueGrowth"),     # y/y
            "freeCashflow":     info.get("freeCashflow"),      # absolute
            "debtToEquity":     info.get("debtToEquity"),      # %
        })
    df = pd.DataFrame(rows).set_index("Ticker")

    # Score: >0 profitability/growth/ROE/FCF, & reasonable leverage
    def score_row(r):
        c = 0
        if pd.notna(r.profitMargins)     and r.profitMargins     > 0:   c +=
        if pd.notna(r.operatingMargins)  and r.operatingMargins  > 0:   c +=
        if pd.notna(r.returnOnEquity)    and r.returnOnEquity    > 0:   c +=
        if pd.notna(r.revenueGrowth)     and r.revenueGrowth     > 0:   c +=
        if pd.notna(r.freeCashflow)      and r.freeCashflow      > 0:   c +=
        if pd.notna(r.debtToEquity)      and r.debtToEquity      < 250: c +=
        return c

    df["fund_score"] = df.apply(score_row, axis=1)

    # --- Soft filter (preferred): keep top X% by score ---
    if TOP_FUND_FRAC is not None:
        # quantile threshold; ties above the cut are kept
        cut = df["fund_score"].quantile(1.0 - (1.0 - float(TOP_FUND_FRAC)))
        fund_ok_vec = (df["fund_score"] >= cut).reindex(symbols).fillna(True
    else:
        # --- Hard cutoff fallback ---
        fund_ok_vec = (df["fund_score"] >= MIN_FUND_SCORE).reindex(symbols).

    return df, fund_ok_vec

if USE_FUNDAMENTALS:
    fund_df, fund_ok_vec = fetch_fundamentals(SYMS)
else:
    fund_df = pd.DataFrame(index=SYMS)
    fund_ok_vec = pd.Series(True, index=SYMS)

# Broadcast to trading calendar (constant over time; refresh quarterly if de
FUND_OK = pd.DataFrame(
    np.repeat(fund_ok_vec.values[None, :], len(px.index), axis=0),
```

```
    index=px.index, columns=SYMS
).astype(int)
```

In [11]:

```
# === Cell F (drop-in) ===
# Soft-score signals (ARIMA, MOM, VWAP, FUND) → continuous gate in [0,1].
# S1/S2: composite of all features; S3: ARIMA-only.
# Daily trading between 6M rebalances; gates apply even on rebalances.

ARIMA_SIGNAL_FREQ    = 'D'    # 'D' or 'W-FRI'
APPLY_GATES_ON_REBAL = True
VWAP_LKBK            = 50
MOM_LKBK_GATE        = None

# ---- Score weights and shaping
W_ARIMA = 0.55
W_MOM   = 0.20
W_VWAP  = 0.15
W_FUND  = 0.10        # used only for S1/S2
SCORE_TEMP   = 0.6    # logistic temperature (higher = softer)
SCORE_THRESH = 0.0    # shift for logistic; 0.0 ≈ neutral

def _zscore(s, clip=3.0):
    """Cross-sectional z-score per date (winsorized)."""
    m = s.mean(axis=1)
    sd = s.std(axis=1).replace(0, np.nan)
    z = (s.sub(m, axis=0)).div(sd, axis=0)
    if clip is not None:
        z = z.clip(-clip, clip)
    return z.fillna(0)

def _logistic(x, temp=SCORE_TEMP, shift=SCORE_THRESH):
    return 1.0 / (1.0 + np.exp(-(x - shift) / max(temp, 1e-6)))

# ---- Prev-day inputs (no look-ahead)
PX_TEST   = px.reindex(test_idx)
price_prev= PX_TEST.shift(1)

VWAP_ROLL = (TP.mul(VOL)).rolling(window=VWAP_LKBK, min_periods=max(10, VWAP
            .div(VOL.rolling(window=VWAP_LKBK, min_periods=max(10, VWAP_LKBK
vwap_prev = VWAP_ROLL.reindex(test_idx).shift(1)

fc_aligned = fc_df.reindex(test_idx)             # forecast for date d (avai
exp_ret    = (fc_aligned / price_prev) - 1.0     # ARIMA expected next-day m

# Feature 1: ARIMA signal (raw = exp_ret)
feat_arima = exp_ret.reindex(test_idx)

# Feature 2: Momentum (MOM_LKBK_GATE)
if MOM_LKBK_GATE:
    mom = px.pct_change(MOM_LKBK_GATE).reindex(test_idx).shift(1)
else:
    mom = pd.DataFrame(0.0, index=test_idx, columns=px.columns)
feat_mom = mom

# Feature 3: VWAP gap (price premium to rolling VWAP)
feat_vwap = (price_prev / vwap_prev - 1.0)
```

```
# Feature 4: Fundamentals (static per name → broadcast over time)
# Requires fund_df["fund_score"] and FUND_OK from fundamentals cell.
fund_score = (fund_df["fund_score"].reindex(px.columns)
              if ('fund_df' in globals() and "fund_score" in fund_df.columns
              else pd.Series(0.0, index=px.columns))
feat_fund = pd.DataFrame(np.tile(fund_score.values, (len(test_idx), 1)),
                         index=test_idx, columns=px.columns)

# ---- Normalize features cross-sectionally per date
Z_arima = _zscore(feat_arima)
Z_mom   = _zscore(feat_mom)
Z_vwap  = _zscore(feat_vwap)
# fundamentals: normalize cross-section once (same each day)
fund_z  = (fund_score - fund_score.mean()) / (fund_score.std() if fund_score
Z_fund  = pd.DataFrame(np.tile(fund_z.values, (len(test_idx), 1)),
                       index=test_idx, columns=px.columns).fillna(0)

# ---- Composite scores
score12 = (W_ARIMA*Z_arima + W_MOM*Z_mom + W_VWAP*Z_vwap + W_FUND*Z_fund).fi
score3  = (W_ARIMA*Z_arima).fillna(0)

# Convert to soft gates in [0,1]
gate12_soft = _logistic(score12)
gate3_soft  = _logistic(score3)

# Optional: resample signals (weekly), then ffill on trading calendar
def _resample_soft(gdf, freq):
    if freq != 'D':
        gdf = gdf.resample(freq).last()
    gdf = gdf.reindex(px.index).ffill()
    if len(test_idx) > 0:
        gdf.loc[gdf.index < test_idx[0]] = 1.0
    return gdf.fillna(1.0)

gate12_full = _resample_soft(gate12_soft, ARIMA_SIGNAL_FREQ)
gate3_full  = _resample_soft(gate3_soft,  ARIMA_SIGNAL_FREQ)

# ---- Simulator: identical to your daily execution engine BUT accepts conti
def simulate_active_between_rebals(prices, base_target_fn, gates_full_df, re
                                   initial_capital=100_000.0, tx_cost=0.0,
                                   trade_weight_bps=1.0, apply_gates_on_reba
                                   window_idx=None):
    if window_idx is None:
        window_idx = prices.index
    cols = list(prices.columns)
    rets = prices.pct_change().fillna(0.0)

    pos_val  = pd.DataFrame(0.0, index=window_idx, columns=cols)
    weights  = pd.DataFrame(0.0, index=window_idx, columns=cols)
    port_val = pd.Series(index=window_idx, dtype=float)
    trade_log = []

    # t0: start at base weights
    t0 = window_idx[0]
    base_w = base_target_fn(t0, prices.loc[t0]).reindex(cols).fillna(0.0)
```

```
    base_w = base_w / max(base_w.sum(), eps)
    pos_val.loc[t0]  = base_w * INITIAL_CAPITAL
    weights.loc[t0]  = base_w
    port_val.iloc[0] = float(pos_val.loc[t0].sum())
    last_base = base_w.copy()

    thresh = trade_weight_bps / 10000.0

    for i in range(1, len(window_idx)):
        prev, d = window_idx[i-1], window_idx[i]
        pv_prev = float(pos_val.loc[prev].sum())
        w_pre   = (pos_val.loc[prev] / max(pv_prev, eps)).fillna(0.0)

        # 1) Update base template on rebalance dates
        if prev in rebalance_dates:
            last_base = base_target_fn(prev, prices.loc[prev]).reindex(cols)
            last_base = last_base / max(last_base.sum(), eps)

        # 2) Build target for day d
        if (prev in rebalance_dates) and (not apply_gates_on_rebal):
            w_tgt_raw = last_base.copy()
        else:
            g = gates_full_df.loc[prev].reindex(cols).fillna(1.0)   # contir
            w_tgt_raw = last_base * g
            s = w_tgt_raw.sum()
            if s <= 0:
                w_tgt_raw = last_base.copy()
        w_tgt = w_tgt_raw / max(w_tgt_raw.sum(), eps)

        # 3) Trade to target (cost on turnover)
        dw = (w_tgt - w_pre)
        gross_turnover = float(dw.abs().sum())
        cost = tx_cost * gross_turnover * pv_prev
        pv_after = max(pv_prev - cost, 0.0)

        # 4) Apply returns for day d
        pos_carry = w_tgt * pv_after
        pos_val.loc[d] = pos_carry * (1.0 + rets.loc[d])
        port_val.iloc[i] = float(pos_val.loc[d].sum())
        weights.loc[d] = (pos_val.loc[d] / max(port_val.iloc[i], eps)).fillr

        trades_today = int((dw.abs() > max(thresh, eps)).sum())
        trade_log.append({"date": prev, "trades": trades_today,
                          "gross_turnover": gross_turnover, "cost": cost})

    trade_df = pd.DataFrame(trade_log).set_index("date") if trade_log else p
    return port_val, weights, trade_df

# ---- Rebalance dates inside the test window
REBAL_DATES_WIN = REBAL_DATES.intersection(test_idx)

# Base target fns must already exist:
#   target_equal_weight(date, prices_row) ; target_cap_weight

# Scenario 1: Equal-Weight + composite (ARIMA+MOM+VWAP+FUND)
S1_val, S1_w, S1_reb = simulate_active_between_rebals(
```

```
    px, target_equal_weight, gate12_full, REBAL_DATES_WIN,
    initial_capital=INITIAL_CAPITAL, tx_cost=TX_COST,
    trade_weight_bps=TRADE_WEIGHT_BPS, apply_gates_on_rebal=APPLY_GATES_ON_F
    window_idx=test_idx
)

# Scenario 2: Cap-Weight + composite (ARIMA+MOM+VWAP+FUND)
S2_val, S2_w, S2_reb = simulate_active_between_rebals(
    px, target_cap_weight, gate12_full, REBAL_DATES_WIN,
    initial_capital=INITIAL_CAPITAL, tx_cost=TX_COST,
    trade_weight_bps=TRADE_WEIGHT_BPS, apply_gates_on_rebal=APPLY_GATES_ON_F
    window_idx=test_idx
)

# Scenario 3: Base case Equal-Weight + ARIMA-only (composite from ARIMA only
S3_val, S3_w, S3_reb = simulate_active_between_rebals(
    px, target_equal_weight, gate3_full, REBAL_DATES_WIN,
    initial_capital=INITIAL_CAPITAL, tx_cost=TX_COST,
    trade_weight_bps=TRADE_WEIGHT_BPS, apply_gates_on_rebal=APPLY_GATES_ON_F
    window_idx=test_idx
)
```

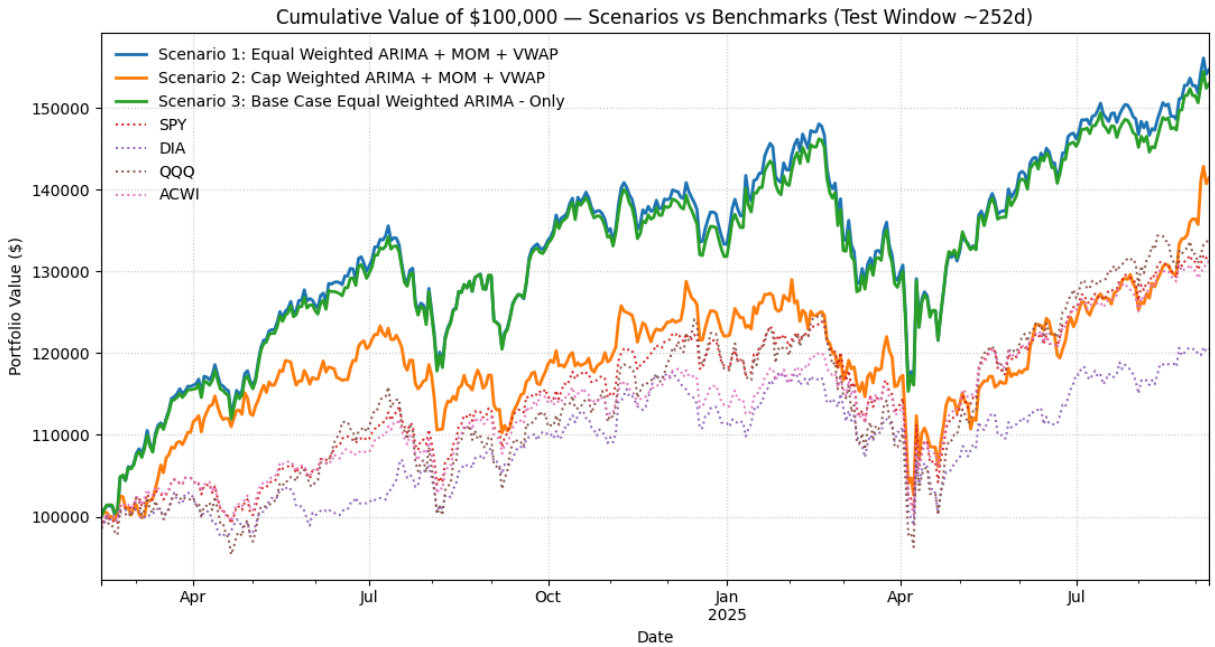
In [12]:

```
# === Cell G (drop-in) ===
import matplotlib.pyplot as plt

# Scenario display names
NAME_S1 = "Scenario 1: Equal Weighted ARIMA + MOM + VWAP"
NAME_S2 = "Scenario 2: Cap Weighted ARIMA + MOM + VWAP"
NAME_S3 = "Scenario 3: Base Case Equal Weighted ARIMA - Only"

window_idx = test_idx
spy_w  = bench_ret_m["SPY"].reindex(window_idx).dropna()

def capm_alpha_annualized(port_returns, rf_series, bench_returns):
    df = pd.concat([
        port_returns.rename("rp"),
        rf_series.reindex(port_returns.index).rename("rf"),
        bench_returns.reindex(port_returns.index).rename("rm")
    ], axis=1).dropna()
    if df.empty:
        return np.nan, np.nan, np.nan
    s_ex = df["rp"] - df["rf"]; m_ex = df["rm"] - df["rf"]
    var_m = m_ex.var()
    beta  = (s_ex.cov(m_ex) / var_m) if var_m > 0 else np.nan
    alpha_d = (s_ex - beta*m_ex).mean()
    r2 = s_ex.corr(m_ex)**2 if (s_ex.std() > 0 and m_ex.std() > 0) else np.r
    return alpha_d * 252.0, beta, r2

def perf_metrics(value_series):
    rets = value_series.pct_change().dropna()
    ann = 252
    ann_ret = (1 + rets).prod() ** (ann/len(rets)) - 1
    ann_vol = rets.std() * np.sqrt(ann)
    curve   = (1 + rets).cumprod()
    maxdd   = (curve / curve.cummax() - 1).min()
    sharpe  = (rets.mean() / (rets.std() + 1e-12)) * np.sqrt(ann) if rets.st
```

```
    win     = (rets > 0).mean()
    end_val = value_series.iloc[-1]
    return {"End Value ($)": end_val, "Ann. Return": ann_ret, "Ann. Vol": ar
            "Sharpe": sharpe, "Max DD": maxdd, "Win Rate": win}

def extra_metrics(port_daily, bench_daily, rf_series, trades_df):
    ann = 252
    # Sortino
    df = pd.concat([port_daily.rename("rp"), rf_series.reindex(port_daily.ir
    if df.empty:
        sortino = np.nan
    else:
        ex = df["rp"] - df["rf"]
        downside = ex[ex < 0]
        dd = downside.std(ddof=0)
        sortino = (ex.mean() / (dd + 1e-12)) * np.sqrt(ann) if dd > 0 else r
    # Calmar
    cagr = (1 + port_daily).prod() ** (ann/len(port_daily)) - 1 if len(port_
    curve_tmp = (1 + port_daily).cumprod()
    mdd = (curve_tmp / curve_tmp.cummax() - 1).min() if len(curve_tmp) > 0 e
    calmar = (cagr / abs(mdd)) if (pd.notna(cagr) and pd.notna(mdd) and mdd
    # IR & TE vs SPY
    rel = pd.concat([port_daily.rename("rp"), bench_daily.rename("rm")], axi
    if rel.empty:
        ir, te, upcap, downcap = np.nan, np.nan, np.nan, np.nan
    else:
        active = rel["rp"] - rel["rm"]
        te = active.std() * np.sqrt(ann)
        ir = (active.mean() / (active.std() + 1e-12)) * np.sqrt(ann) if acti
        up_mask, down_mask = rel["rm"] > 0, rel["rm"] < 0
        def cap(mask):
            if mask.sum() == 0: return np.nan
            rp = (1 + rel.loc[mask, "rp"]).prod() - 1
            rm = (1 + rel.loc[mask, "rm"]).prod() - 1
            return (rp / rm) if rm != 0 else np.nan
        upcap, downcap = cap(up_mask), cap(down_mask)

    gross_to_pct = float(trades_df["gross_turnover"].sum()) * 100.0 if (trac
    trades_num   = int(trades_df["trades"].sum()) if (trades_df is not None

    return {
        "Sortino": sortino, "Calmar": calmar,
        "Information Ratio": ir, "Tracking Error": te,
        "Up Capture": upcap, "Down Capture": downcap,
        "Gross Turnover (%)": gross_to_pct, "Total Trades (#)": trades_num
    }

# Window-rebased curves
def curve_from_value(v, idx):
    r = v.reindex(idx).pct_change().dropna()
    out = (1 + r).cumprod() * INITIAL_CAPITAL
    out.index = idx[1:]
    return pd.concat([pd.Series([INITIAL_CAPITAL], index=[idx[0]]), out])

S1_curve = curve_from_value(S1_val, window_idx)
S2_curve = curve_from_value(S2_val, window_idx)
```

```
S3_curve = curve_from_value(S3_val, window_idx)
bench_curves_window = (1.0 + bench_ret_m.reindex(window_idx)).cumprod() * IN

# Plot
fig, ax = plt.subplots(figsize=(11, 6))
S1_curve.rename(NAME_S1).plot(ax=ax, linewidth=2)
S2_curve.rename(NAME_S2).plot(ax=ax, linewidth=2)
S3_curve.rename(NAME_S3).plot(ax=ax, linewidth=2)
for col in bench_curves_window.columns:
    bench_curves_window[col].plot(ax=ax, linewidth=1.4, linestyle=":")
ax.set_title("Cumulative Value of $100,000 — Scenarios vs Benchmarks (Test W
ax.set_xlabel("Date"); ax.set_ylabel("Portfolio Value ($)")
ax.legend(loc="best", frameon=False); ax.grid(True, linestyle=":", alpha=0.6
plt.tight_layout(); plt.show()

# Returns (window)
S1_ret = S1_curve.reindex(window_idx).pct_change().dropna()
S2_ret = S2_curve.reindex(window_idx).pct_change().dropna()
S3_ret = S3_curve.reindex(window_idx).pct_change().dropna()

# CAPM vs SPY
alpha_S1, beta_S1, r2_S1 = capm_alpha_annualized(S1_ret, rf_daily, spy_w)
alpha_S2, beta_S2, r2_S2 = capm_alpha_annualized(S2_ret, rf_daily, spy_w)
alpha_S3, beta_S3, r2_S3 = capm_alpha_annualized(S3_ret, rf_daily, spy_w)

rows = {
    NAME_S1: perf_metrics(S1_curve),
    NAME_S2: perf_metrics(S2_curve),
    NAME_S3: perf_metrics(S3_curve),
}
for col in bench_curves_window.columns:
    rows[col] = perf_metrics(bench_curves_window[col])

summary_df = pd.DataFrame(rows).T

# CAPM
summary_df.loc[NAME_S1, ["CAPM Alpha (annual)", "Beta", "R²"]] = [alpha_S1,
summary_df.loc[NAME_S2, ["CAPM Alpha (annual)", "Beta", "R²"]] = [alpha_S2,
summary_df.loc[NAME_S3, ["CAPM Alpha (annual)", "Beta", "R²"]] = [alpha_S3,

# Manager-style extras + daily trade stats
S1_ex = extra_metrics(S1_ret, spy_w, rf_daily, S1_reb)
S2_ex = extra_metrics(S2_ret, spy_w, rf_daily, S2_reb)
S3_ex = extra_metrics(S3_ret, spy_w, rf_daily, S3_reb)
for name, ex in [(NAME_S1,S1_ex),(NAME_S2,S2_ex),(NAME_S3,S3_ex)]:
    for k,v in ex.items():
        summary_df.loc[name, k] = v

# Pretty formatting
pct_cols = ["Ann. Return","Ann. Vol","Max DD","Win Rate","CAPM Alpha (annual
for c in pct_cols:
    summary_df[c] = (100.0 * summary_df[c].astype(float)).map(lambda v: f"{v
for c in ["Sharpe","Sortino","Calmar","Information Ratio"]:
    summary_df[c] = summary_df[c].map(lambda v: f"{float(v):.2f}" if pd.notr
summary_df["Beta"]   = summary_df["Beta"].map(lambda v: f"{float(v):.3f}" if
summary_df["R²"]     = summary_df["R²"].map(lambda v: f"{float(v):.3f}" if p
```

```
summary_df["End Value ($)"] = summary_df["End Value ($)"].map(lambda v: f"${
summary_df["Total Trades (#)"] = summary_df["Total Trades (#)"].fillna("").a

print("=== Scenarios & Benchmarks Summary (Test Window ≈ 252 trading days) =
display(summary_df)
```




```
=== Scenarios & Benchmarks Summary (Test Window ≈ 252 trading days) ===
```

	End Value ($)	Ann. Return	Ann. Vol	Sharpe	Max DD	Win Rate	CAPM Alpha (annual)	Beta	R^2	S
Scenario 1: Equal Weighted ARIMA + MOM + VWAP	$154,691	30.84%	20.72%	1.40	-21.70%	53.79%	8.75%	1.059	0.766	
Scenario 2: Cap Weighted ARIMA + MOM + VWAP	$141,427	23.81%	19.35%	1.20	-20.42%	53.79%	4.56%	0.951	0.708	
Scenario 3: Base Case Equal Weighted ARIMA - Only	$152,946	29.93%	20.47%	1.38	-21.13%	53.06%	8.17%	1.047	0.768	
SPY	$131,957	19.65%	17.13%	1.13	-18.76%	55.50%	nan%	NA	NA	
DIA	$120,541	13.15%	14.88%	0.90	-15.95%	52.08%	nan%	NA	NA	
QQQ	$134,110	20.99%	21.66%	0.99	-22.77%	55.50%	nan%	NA	NA	
ACWI	$131,352	19.47%	15.29%	1.24	-16.55%	52.81%	nan%	NA	NA	